

Equivariant Diffusion for 3D Molecule Generation on QM9

A line-by-line walkthrough of the implementation

Code documentation

Abstract

This document explains an implementation of an $E(3)$ -equivariant denoising diffusion probabilistic model that generates small organic molecules as 3D point clouds — atom types *and* Cartesian coordinates jointly — trained on QM9. It is a compact reimplementation of EDM (Hooeboom et al., 2022) with an EGNN (Satorras et al., 2021) denoiser, depending only on `torch` and `numpy`.

The emphasis throughout is on **tensor shapes**: every array is annotated with its dimensions, and each transformation states what the shape was before and after. All shapes, parameter counts and schedule values quoted here were measured by instrumenting the running code, not derived on paper.

Contents

1	Notation and the dimension glossary	2
2	The data pipeline: <code>qm9_diffusion/data.py</code>	2
2.1	From the raw release to a padded cache	2
2.2	What one training example looks like	3
2.3	The size distribution $p(N_{\text{atoms}})$	3
3	The zero-centre-of-mass subspace	4
4	The denoiser: <code>qm9_diffusion/egnn.py</code>	5
4.1	Why an EGNN and not an MLP	5
4.2	Input embedding: where the timestep enters	5
4.3	One EGNN layer, shape by shape	6
4.4	Output heads	7
4.5	Parameter budget	8
5	The diffusion model: <code>qm9_diffusion/diffusion.py</code>	8
5.1	The forward (noising) process	8
5.2	The noise schedule	8
5.3	Scaling of the two modalities	9
5.4	The training loss	9
6	Training: <code>train.py</code>	10
6.1	The loop	10
6.2	Per-epoch evaluation	11
7	Sampling: <code>EquivariantDiffusion.sample</code>	11
7.1	The reverse process	11
7.2	The chain and the final decode	12
7.3	A closed-form check of the sampler	12

8	Evaluation: qm9_diffusion/stability.py	13
9	Masking: the invariant to preserve	13
10	Computational cost	14
10.1	Where the memory goes	14
10.2	Measured throughput	14
11	Summary: one forward pass end to end	14

1 Notation and the dimension glossary

Six symbols determine every shape in the codebase. Fix them once and the rest of the document follows.

Symbol	Meaning	Code name	Typical value
B	batch size (molecules per batch)	<code>batch_size</code>	64
N	padded atom count (max in split)	<code>max_n</code>	29 with H, 9 without
K	number of atom types	<code>num_types</code>	5 (H,C,N,O,F); 4 without H
H	hidden width of the EGNN	<code>hidden</code>	192
L	number of EGNN layers	<code>n_layers</code>	6
T	diffusion timesteps	<code>timesteps</code>	1000

A molecule is represented *densely*: every molecule in a batch is padded to N atoms and carries a binary mask marking which slots are real.

Tensor	Shape	Contents
<code>x</code>	$[B, N, 3]$	atom coordinates in Ångström, zeros in padded slots
<code>h</code>	$[B, N, K]$	one-hot atom types, all-zero rows in padded slots
<code>mask</code>	$[B, N]$	1.0 for a real atom, 0.0 for padding

The mask is the load-bearing part of the dense design. It appears in every sum, every mean and every layer output; a single missing multiplication by it lets padded slots contribute to a molecule’s chemistry. Section 9 collects the places where this matters.

Why N differs between modes. N is not a constant of the model but of the *data split*: `QM9Dataset` truncates its padded arrays to the largest molecule it actually contains. With hydrogens that is 29; with `--remove-h` the heavy-atom skeletons top out around 9. Since the cost of a forward pass grows as N^2 (Section 10), this single number accounts for the $\sim 6\times$ speed difference between the two modes.

2 The data pipeline: [qm9_diffusion/data.py](#)

2.1 From the raw release to a padded cache

QM9 is 133,885 small organic molecules over $\{\text{H, C, N, O, F}\}$, each with a DFT-relaxed geometry. The loader needs only types and coordinates, so it discards everything else and caches one `.npz`:

Stage	Representation	Shape	dtype
raw	gdb9.sdf, one text record per molecule	—	text
parsed	Python list of per-molecule arrays	$M \times (n_i, 3)$	float32
cached	positions	[133885, 29, 3]	float32
	atom_types	[133885, 29]	int8
	num_atoms	[133885]	int16

Molecule i occupies rows $0 \dots n_i - 1$ of its slice; rows $n_i \dots 28$ are zero. The cache is ≈ 50 MB and is built once.

The V2000 atom line. `_parse_sdf` reads coordinates by fixed column offsets rather than by splitting on whitespace, because the MOL format is column-oriented and the fields can abut:

```

1 sym = line[31:34].strip() # element symbol
2 pos[i] = (float(line[0:10]), # x, columns 1-10
3          float(line[10:20]), # y, columns 11-20
4          float(line[20:30])) # z, columns 21-30

```

Listing 1: Fixed-width parse of one SDF atom line `data.py`

That the resulting geometries reproduce the published stability statistics exactly (Section 8) is the evidence that this parse is correct.

2.2 What one training example looks like

`QM9Dataset.__getitem__` converts stored integer types into the dense masked form the model consumes:

```

1 n = int(self.num_atoms[i]) # scalar: true atom count
2 mask = torch.zeros(self.max_n) # [N]
3 mask[:n] = 1.0 # [N], n ones then zeros
4 one_hot = F.one_hot(self.atom_types[i], self.num_types).float() # [N] -> [N,K]
5 return {
6     "x": self.positions[i] * mask[:, None], # [N,3] * [N,1] -> [N,3]
7     "h": one_hot * mask[:, None], # [N,K] * [N,1] -> [N,K]
8     "mask": mask, # [N]
9 }

```

Listing 2: `QM9Dataset.__getitem__` — integer types to masked one-hot

Two shape details deserve attention:

- `mask[:, None]` promotes $[N]$ to $[N, 1]$ so it broadcasts against both $[N, 3]$ and $[N, K]$. The same idiom recurs throughout the model as `node_mask.unsqueeze(-1)`.
- The one-hot of a padded slot would be e_0 (type index 0), i.e. a spurious hydrogen. Multiplying by the mask replaces it with the all-zero row, which is what makes padded atoms genuinely typeless.

The `DataLoader` then stacks B such dictionaries, prepending the batch axis and producing exactly the $[B, N, 3]$, $[B, N, K]$, $[B, N]$ triple of Section 1.

2.3 The size distribution $p(N_{\text{atoms}})$

At generation time the model must be told *how many* atoms to generate — the diffusion process itself does not create or destroy nodes. The empirical histogram supplies that:

```

1 hist = torch.bincount(self.num_atoms, minlength=self.max_n + 1).float() # [N+1]
2 return hist / hist.sum() # [N+1]

```

Listing 3: `size_histogram` — an empirical prior over molecule size

Index k holds $p(n_{\text{atoms}} = k)$, hence length $N + 1$ rather than N . Sampling proceeds by drawing sizes and turning them into a mask:

```

1 sizes = torch.multinomial(size_hist, n_samples, replacement=True) # [B]
2 mask = (torch.arange(max_n)[None, :] < sizes[:, None]).float() # [1,N] vs [B,1] -> [B,N]

```

The comparison broadcasts a row vector of positions against a column vector of lengths, yielding a left-aligned boolean mask in one operation.

3 The zero-centre-of-mass subspace

This is the one piece of geometry that must be understood before the model makes sense, so it comes before the network.

The problem. We want the learned density $p(\mathbf{x})$ over $\mathbf{x} \in \mathbb{R}^{n \times 3}$ to be invariant to rigid motions: a molecule and its translated copy are the same molecule. But no probability density on $\mathbb{R}^{n \times 3}$ can be translation invariant. If $p(\mathbf{x}) = p(\mathbf{x} + t\mathbf{1})$ for every $t \in \mathbb{R}^3$, then p is constant along 3 unbounded directions and cannot integrate to 1.

The fix. Quotient out the offending directions. Work on the linear subspace of configurations whose centre of mass is the origin,

$$\mathcal{X}_0 = \left\{ \mathbf{x} \in \mathbb{R}^{n \times 3} : \sum_{i=1}^n \mathbf{x}_i = \mathbf{0} \right\}, \quad \dim \mathcal{X}_0 = 3(n - 1). \quad (1)$$

A density on $\mathcal{X}_0$ is normalisable, and pushing it forward by "place the centre of mass anywhere" gives the translation-invariant object we actually want. Every coordinate quantity in the code therefore lives in $\mathcal{X}_0$: the data, the noise, and the network’s coordinate output.

The projection. It is a mean subtraction, but a *masked* one: the mean must be taken over the n_b real atoms of molecule b , not over all N padded slots.

```

1 def remove_mean(x, node_mask): # x [B,N,3], node_mask [B,N]
2     nm = node_mask.unsqueeze(-1) # [B,N] -> [B,N,1]
3     n = node_mask.sum(1).view(-1, 1, 1).clamp(min=1.0) # [B,N] -> [B] -> [B,1,1]
4     mean = (x * nm).sum(1, keepdim=True) / n # [B,N,3] -> [B,1,3]
5     return (x - mean) * nm # [B,1,3] broadcasts -> [B,N,3]

```

Listing 4: `remove_mean` — projection onto $\mathcal{X}_0$ `diffusion.py`

Shape trace: $[B, N, 3] \rightarrow \text{sum over the atom axis} \rightarrow [B, 1, 3] \rightarrow \text{broadcast back to } [B, N, 3]$. The `keepdim=True` is what makes the subtraction broadcast without an explicit `unsqueeze`. The `clamp(min=1.0)` guards against a division by zero for an all-padding row.

Noise must be projected too. Gaussian noise added to a point in $\mathcal{X}_0$ has to stay in $\mathcal{X}_0$, so the sampler never uses raw `randn` for coordinates:

```

1 def sample_zero_com_noise(shape, node_mask, device):
2     return remove_mean(torch.randn(shape, device=device), node_mask) # [B,N,3]

```

This projected noise is *not* standard normal: it has variance $\frac{n-1}{n}$ per coordinate, a fact used to check the sampler in Section 7.3.

4 The denoiser: `qm9_diffusion/egnn.py`

The network’s job is a regression: given a noisy molecule and a timestep, predict the noise that was added. It returns two tensors, $\hat{\epsilon}_x [B, N, 3]$ and $\hat{\epsilon}_h [B, N, K]$.

4.1 Why an EGNN and not an MLP

Rotating a molecule rotates the noise that was added to it. A generic network would have to learn this from data, spending capacity on a symmetry we already know. An EGNN builds it in. Writing R for a rotation/reflection in $O(3)$, the network satisfies

$$\hat{\epsilon}_x(\mathbf{x}R, \mathbf{h}, t) = \hat{\epsilon}_x(\mathbf{x}, \mathbf{h}, t) R, \quad \hat{\epsilon}_h(\mathbf{x}R, \mathbf{h}, t) = \hat{\epsilon}_h(\mathbf{x}, \mathbf{h}, t), \quad (2)$$

i.e. the coordinate output is *equivariant* and the type output is *invariant*. The two structural facts that produce this:

1. messages are functions of squared interatomic distances $\|\mathbf{x}_i - \mathbf{x}_j\|^2$, which are $O(3)$ -invariant;
2. coordinates are updated only along relative-position vectors $\mathbf{x}_i - \mathbf{x}_j$, which are $O(3)$ -equivariant.

Scalars times equivariant vectors stay equivariant, and the composition of equivariant layers is equivariant, so the property holds for the whole stack. `test_smoke.py` verifies it numerically to 5×10^{-7} .

4.2 Input embedding: where the timestep enters

```
1 b, n, _ = x.shape # B, N
2 nm = node_mask.unsqueeze(-1) # [B,N] -> [B,N,1]
3 eye = torch.eye(n).view(1, n, n, 1) # [N,N] -> [1,N,N,1]
4 edge_mask = (nm.unsqueeze(2) * nm.unsqueeze(1)) * (1.0 - eye) # -> [B,N,N,1]
5
6 t_node = t.view(b, 1, 1).expand(-1, n, 1) # [B] -> [B,1,1] -> [B,N,1]
7 hidden = self.embed(torch.cat([h, t_node], -1)) * nm # [B,N,K+1] -> [B,N,H]
```

Listing 5: `EGNNDynamics.forward` — masks and embedding

Three shape manipulations, each worth naming:

Expression		Before	After	Purpose
<code>nm.unsqueeze(2)</code>	<code>* nm.unsqueeze(1)</code>	<code>[B,N,1]</code>	<code>[B,N,N,1]</code>	outer product of the masks: edge (i,j) is live iff both ends are real
<code>* (1.0 - eye)</code>		<code>[B,N,N,1]</code>	<code>[B,N,N,1]</code>	delete the self-loops
<code>t.view(b,1,1)</code>	<code>.expand(-1,n,1)</code>	<code>[B]</code>	<code>[B,N,1]</code>	broadcast the scalar time to every node

The time conditioning is deliberately minimal: t is a single scalar in $[0, 1]$ concatenated as one extra input channel, so the embedding is a linear map $\mathbb{R}^{K+1} \rightarrow \mathbb{R}^H$ ($6 \rightarrow 192$ by default, only 1,344 parameters). There is no sinusoidal Fourier embedding here — a design simplification relative to image diffusion models, where the timestep is usually embedded richly.

`expand` rather than `repeat` is used because it creates a *view* with stride 0 along the atom axis; no $B \times N$ block of memory is written for what is logically one number per molecule.

4.3 One EGNN layer, shape by shape

This is the core of the model. The layer maps $(\mathbf{h}, \mathbf{x}) \in [\mathbf{B}, \mathbf{N}, \mathbf{H}] \times [\mathbf{B}, \mathbf{N}, \mathbf{3}]$ to a tensor pair of the same two shapes, but the intermediate quantities live on *edges* and carry a fourth axis. With $B = 64, N = 29, H = 192$ the trace is:

Line	Quantity	Shape	Concrete
in	$\mathbf{h}$	$[\mathbf{B}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 192)$
in	$\mathbf{x}$	$[\mathbf{B}, \mathbf{N}, \mathbf{3}]$	$(64, 29, 3)$
1	$\mathbf{diff}$	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{3}]$	$(64, 29, 29, 3)$
2	$\mathbf{d2}$	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{1}]$	$(64, 29, 29, 1)$
3	$\mathbf{norm}$	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{1}]$	$(64, 29, 29, 1)$
4	$\mathbf{hi}$ (expand)	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 29, 192)$
5	$\mathbf{hj}$ (expand)	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 29, 192)$
6	edge_mlp input	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, 2\mathbf{H}+1]$	$(64, 29, 29, 385)$
7	$\mathbf{m}$ (message)	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 29, 192)$
8	$\mathbf{att}$ (gate)	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{1}]$	$(64, 29, 29, 1)$
9	$\mathbf{m} * \mathbf{att} * \mathbf{edge_mask}$	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 29, 192)$
10	coord_mlp($\mathbf{m}$)	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{1}]$	$(64, 29, 29, 1)$
11	$\mathbf{trans}$	$[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{3}]$	$(64, 29, 29, 3)$
12	$\mathbf{n_edges}$	$[\mathbf{B}, \mathbf{N}, \mathbf{1}]$	$(64, 29, 1)$
13	$\mathbf{agg}$	$[\mathbf{B}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 192)$
14	node_mlp input	$[\mathbf{B}, \mathbf{N}, 2\mathbf{H}]$	$(64, 29, 384)$
out	$\mathbf{h}$	$[\mathbf{B}, \mathbf{N}, \mathbf{H}]$	$(64, 29, 192)$
out	$\mathbf{x}$	$[\mathbf{B}, \mathbf{N}, \mathbf{3}]$	$(64, 29, 3)$

The pattern is a round trip: **node axis** $\rightarrow$ **edge axis** $\rightarrow$ **node axis**. Lines 1–9 lift $[\mathbf{B}, \mathbf{N}, \cdot]$ to $[\mathbf{B}, \mathbf{N}, \mathbf{N}, \cdot]$; lines 11–13 contract the third axis away with a sum. The code:

```

1 diff = x.unsqueeze(2) - x.unsqueeze(1) # [B,N,1,3] - [B,1,N,3] -> [B,N,N,3]
2 d2 = (diff**2).sum(-1, keepdim=True) # -> [B,N,N,1]
3 norm = (d2 + 1e-8).sqrt() # -> [B,N,N,1]
4
5 hi = h.unsqueeze(2).expand(-1, -1, n, -1) # [B,N,1,H] -> [B,N,N,H] (sender)
6 hj = h.unsqueeze(1).expand(-1, n, -1, -1) # [B,1,N,H] -> [B,N,N,H] (receiver)
7 m = self.edge_mlp(torch.cat([hi, hj, d2], dim=-1)) # [B,N,N,2H+1] -> [B,N,N,H]
8 m = m * self.att_mlp(m) * edge_mask # [B,N,N,H] * [B,N,N,1] * [B,N,N,1]
9
10 trans = diff / (norm + 1.0) * torch.tanh(self.coord_mlp(m)) * self.coords_range
11 trans = trans * edge_mask # [B,N,N,3]
12 n_edges = edge_mask.sum(dim=2).clamp(min=1.0) # [B,N,N,1] -> [B,N,1]
13 x = x + (trans.sum(dim=2) / n_edges) * node_mask # [B,N,N,3] -> [B,N,3]
14
15 agg = m.sum(dim=2) / n_edges # [B,N,N,H] -> [B,N,H]
16 h = h + self.node_mlp(torch.cat([h, agg], dim=-1)) # [B,N,2H] -> [B,N,H]
17 return h * node_mask, x * node_mask

```

Listing 6: EquivariantConv.forward

The broadcast that builds the edge axis. $\mathbf{x}.\text{unsqueeze}(2)$ is $[\mathbf{B}, \mathbf{N}, \mathbf{1}, \mathbf{3}]$ and $\mathbf{x}.\text{unsqueeze}(1)$ is $[\mathbf{B}, \mathbf{1}, \mathbf{N}, \mathbf{3}]$; subtracting them broadcasts both to $[\mathbf{B}, \mathbf{N}, \mathbf{N}, \mathbf{3}]$, and entry (b, i, j) of the result

holds $\mathbf{x}_i - \mathbf{x}_j$. This one line materialises all N^2 ordered pairs. The same asymmetry names the two expanded copies of $\mathbf{h}$: $\mathbf{h}_i$ varies along axis 1 and $\mathbf{h}_j$ along axis 2, so the concatenation at position (i, j) is $[\mathbf{h}_i, \mathbf{h}_j, d_{ij}^2]$.

The equations. In standard EGNN notation, with ϕ_e, ϕ_h, ϕ_x the three MLPs and ϕ_a the gate:

$$\mathbf{m}_{ij} = \phi_a(\tilde{\mathbf{m}}_{ij}) \tilde{\mathbf{m}}_{ij}, \quad \tilde{\mathbf{m}}_{ij} = \phi_e(\mathbf{h}_i, \mathbf{h}_j, \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad (3)$$

$$\mathbf{x}'_i = \mathbf{x}_i + \frac{1}{|\mathcal{N}_i|} \sum_{j \neq i} \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\| + 1} \gamma \tanh(\phi_x(\mathbf{m}_{ij})), \quad (4)$$

$$\mathbf{h}'_i = \mathbf{h}_i + \phi_h\left(\mathbf{h}_i, \frac{1}{|\mathcal{N}_i|} \sum_{j \neq i} \mathbf{m}_{ij}\right). \quad (5)$$

Design choices inside the layer.

The +1 in the denominator.

$\text{diff} / (\text{norm} + 1.0)$ is not a unit vector. Dividing by $\|\mathbf{x}_i - \mathbf{x}_j\|$ alone would make the update blow up for nearby atoms; the offset bounds the factor while keeping direction, and it is still equivariant because it scales an equivariant vector by an invariant scalar.

$\gamma \tanh(\cdot)$.

`coords_range` is passed as $\gamma = 15/L = 2.5 \text{ \AA}$, so a single layer can displace an atom by at most $2.5 \text{ \AA}$ and the whole stack by at most $15 \text{ \AA}$. Without this bound the geometry can diverge early in training, and because distances feed back into the next layer’s messages, a diverged geometry produces garbage messages rather than merely a large loss.

Small initialisation of the coordinate head.

`xavier_uniform(..., gain=0.001)` makes each layer start as approximately the identity on coordinates, so the model begins near "predict no displacement" and learns geometry gradually.

Mean rather than sum aggregation.

Dividing by `n_edges` keeps activation magnitudes independent of molecule size. Because `n_edges` is computed from `edge_mask`, the denominator is the true neighbour count $n_b - 1$, not $N - 1$: a 5-atom molecule in a batch padded to 29 is normalised by 4.

The gate.

`att_mlp` is a $\text{Linear}(H, 1) + \sigma$ producing one scalar in $(0, 1)$ per edge. All N^2 pairs are present in a dense graph, most of them non-bonded; the gate lets the network suppress them instead of being forced to attend to everything.

A pitfall that this code hit. Line 3 reads `(d2 + 1e-8).sqrt()` rather than `d2.sqrt()`. Self-pairs ($i = j$) and padding-padding pairs both have `diff=0`, hence `d2=0`, and $\frac{d}{du}\sqrt{u} \rightarrow \infty$ as $u \rightarrow 0$. The forward pass looks fine — those entries are killed by `edge_mask` — but the backward pass computes $\infty \times 0 = \text{NaN}$ and the loss becomes NaN on the first step. Masking an output does not protect its gradient; the guard has to be inside the square root.

4.4 Output heads

```
1 eps_x = (x - x0) * nm # [B,N,3]
2 eps_x = eps_x - eps_x.sum(1, keepdim=True) / node_mask.sum(1).view(b,1,1)
3 eps_x = eps_x * nm # [B,N,3], zero-CoM
4 eps_h = self.decode(hidden) * nm # [B,N,H] -> [B,N,K]
```

Listing 7: From the last layer to $(\hat{\mathbf{e}}_x, \hat{\mathbf{e}}_h)$

Two things happen here.

1. **The coordinate prediction is a displacement**, $\mathbf{x}^{(L)} - \mathbf{x}^{(0)}$, not the final coordinates. This is what makes the output transform correctly: both terms rotate together, so their difference does, whereas an absolute position would also have to track the input’s placement.
2. **It is re-projected onto $\mathcal{X}_0$** . The target noise lives in the subspace, so the prediction must too — otherwise the model wastes capacity on a component that can never reduce the loss, and the sampler accumulates centre-of-mass drift over T steps. Verified to 1.5×10^{-11} in the tests.

Note $\hat{\mathbf{e}}_h$ is a free real-valued tensor in $\mathbb{R}^{B \times N \times K}$, not a probability vector: there is no softmax. It predicts Gaussian noise on the relaxed one-hot representation, which is why the loss below is a plain MSE.

4.5 Parameter budget

Measured for $H = 192$, $L = 6$, $K = 5$:

Module	Shape of the map	Params	$\times L$
embed	$K+1 \rightarrow H$ ($6 \rightarrow 192$)	1,344	—
edge_mlp	$2H+1 \rightarrow H \rightarrow H$ ($385 \rightarrow 192 \rightarrow 192$)	111,168	667,008
att_mlp	$H \rightarrow 1$	193	1,158
node_mlp	$2H \rightarrow H \rightarrow H$ ($384 \rightarrow 192 \rightarrow 192$)	110,976	665,856
coord_mlp	$H \rightarrow H \rightarrow 1$ ($192 \rightarrow 192 \rightarrow 1$)	37,249	223,494
decode	$H \rightarrow H \rightarrow K$ ($192 \rightarrow 192 \rightarrow 5$)	38,021	—
Total		1,596,881	

97.5% of the parameters sit in the L repeated layers, split almost evenly between `edge_mlp` and `node_mlp`. The diffusion wrapper adds no parameters — its schedules are registered as *buffers*, so they move with `.to(device)` and are saved in checkpoints but are not optimised.

5 The diffusion model: [qm9_diffusion/diffusion.py](#)

5.1 The forward (noising) process

Write $\mathbf{z}_0 = [\mathbf{x}, \mathbf{h}]$ for a clean molecule. The forward process is the usual variance-preserving Gaussian diffusion,

$$q(\mathbf{z}_t | \mathbf{z}_0) = \mathcal{N}(\mathbf{z}_t; \alpha_t \mathbf{z}_0, \sigma_t^2 \mathbf{I}), \quad \text{implemented as} \quad \mathbf{z}_t = \alpha_t \mathbf{z}_0 + \sigma_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}), \quad (6)$$

with $\alpha_t^2 + \sigma_t^2 = 1$, so α_t decays from ≈ 1 to ≈ 0 as $t : 0 \rightarrow T$. Coordinates and types are noised jointly with the *same* (α_t, σ_t) , but the coordinate noise is drawn in $\mathcal{X}_0$ and the type noise in the full $\mathbb{R}^K$.

5.2 The noise schedule

`polynomial_schedule` returns the array $\bar{\alpha}_t = \alpha_t^2$ for $t = 0, \dots, T$ — length $T+1$, i.e. 1001 entries for $T = 1000$:

$$\bar{\alpha}_t = (1 - 2s) \cdot \text{clip} \left[\left(1 - (t/T')^2 \right)^2 \right] + s, \quad s = 10^{-5}, \quad T' = T + 1. \quad (7)$$

`_clip_noise_schedule` bounds the per-step ratio $\bar{\alpha}_t / \bar{\alpha}_{t-1} \geq 0.001$ so that no single step annihilates the signal, then re-accumulates with `cumprod`. The affine map by s keeps $\bar{\alpha}$ strictly inside $(0, 1)$: at $t = 0$ a tiny amount of noise remains (which is why sampling ends with an explicit denoise at $t = 0$ rather than just reading off $\mathbf{z}_0$), and at $t = T$ a tiny amount of signal remains, keeping $\alpha_T > 0$ so the division in the sampler is safe.

Measured values, with the signal-to-noise ratio $\text{SNR} = \alpha_t^2 / \sigma_t^2$:

t	α_t	σ_t	SNR
0	0.999995	0.003162	1.0×10^5
100	0.989995	0.141101	49.2
500	0.749999	0.661439	1.29
900	0.190024	0.981779	0.0375
1000	0.003163	0.999995	1.0×10^{-5}

5.3 Scaling of the two modalities

```

1 def normalize(self, x, h): return x, h * self.h_scale # h_scale = 0.25
2 def unnormalize(self, x, h): return x, h / self.h_scale

```

Shapes are untouched; only the magnitude of h changes. Coordinates have a per-component standard deviation of about 1.4 Å in QM9, whereas a one-hot entry is exactly 0 or 1 — and it is a *discontinuous* target. Left unscaled, the type channels dominate the gradient and the model under-fits geometry. Multiplying by 0.25 puts the two modalities on comparable footing. Because the argmax used at decoding time is invariant to a positive rescaling, this choice affects only the loss balance, never the decoded label.

5.4 The training loss

```

1 b = x.shape[0]
2 x, h = self.normalize(x, h) # [B,N,3], [B,N,K]
3 x = remove_mean(x, node_mask) # project data into X_0
4
5 t_int = torch.randint(0, self.timesteps + 1, (b,)) # [B] integers in {0..T}
6 alpha = self.alphas[t_int].view(b, 1, 1) # [B] -> [B,1,1]
7 sigma = self.sigmas[t_int].view(b, 1, 1) # [B] -> [B,1,1]
8
9 eps_x = sample_zero_com_noise(x.shape, node_mask, x.device) # [B,N,3] in X_0
10 eps_h = torch.randn_like(h) * node_mask.unsqueeze(-1) # [B,N,K]
11
12 z_x = alpha * x + sigma * eps_x # [B,1,1]*[B,N,3] -> [B,N,3]
13 z_h = alpha * h + sigma * eps_h # [B,1,1]*[B,N,K] -> [B,N,K]
14
15 pred_x, pred_h = self.dynamics(t_int.float() / self.timesteps, z_x, z_h, node_mask)
16
17 err = ((pred_x - eps_x)**2).sum() + ((pred_h - eps_h)**2).sum() # -> scalar
18 n_atoms = node_mask.sum() # -> scalar
19 dof = 3 * (n_atoms - b) + self.num_types * n_atoms # -> scalar
20 return err / dof

```

Listing 8: EquivariantDiffusion.loss

Shape observations:

- `alphas` is indexed by a `[B]` integer tensor, giving a `[B]` result, reshaped to `[B,1,1]` so it broadcasts over both the atom and feature axes. Each molecule in the batch gets its *own* timestep — this is what makes one pass over the data cover the whole of $[0, T]$.
- `t_int.float() / self.timesteps` converts integer timesteps to the $[0, 1]$ scalars the network expects, shape `[B]`.
- Both squared errors reduce to a scalar over *all* axes at once. Padded slots contribute exactly 0: `pred` is masked by the network and `eps` is masked at construction.

The normalising constant. The loss is the eps-prediction objective with weight $w(t) = 1$,

$$\mathcal{L} = \frac{\sum_b \|\hat{\epsilon}_x^{(b)} - \epsilon_x^{(b)}\|^2 + \|\hat{\epsilon}_h^{(b)} - \epsilon_h^{(b)}\|^2}{\underbrace{3(\sum_b n_b - B)}_{\text{coordinate d.o.f.}} + \underbrace{K \sum_b n_b}_{\text{type d.o.f.}}} \quad (8)$$

The $-B$ is the geometry of Section 3 showing up in the arithmetic: each molecule contributes $3(n_b - 1)$ coordinate degrees of freedom, not $3n_b$, because the centre-of-mass constraint removes three. Dividing by the true dimension makes the reported number a per-dimension mean, comparable across batches with different molecule sizes — and hence comparable between the training and validation logs.

What is omitted. This is EDM’s L_{simple} only. There is no variational bound / negative log-likelihood term, no L_0 reconstruction term, and no t -dependent reweighting. That is the standard practical choice — the paper reports that $w(t) = 1$ trains better samples than the true ELBO weighting — but it means this code cannot report likelihoods.

6 Training: `train.py`

6.1 The loop

```

1 for batch in train_loader:
2     x, h, mask = (batch[k].to(device) for k in ("x", "h", "mask")) # [B,N,3] [B,N,K] [B,N]
3     loss = model.loss(x, h, mask) # scalar
4     opt.zero_grad(set_to_none=True)
5     loss.backward()
6     grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), args.clip_grad)
7     opt.step()
8     ema.update(model)

```

Listing 9: The inner training step

Four details carry weight here.

Gradient clipping is not optional.

The observed gradient norm on the first iteration is ≈ 530 , falling to ≈ 100 within one epoch. Clipping at 1.5 is what keeps the early updates from destroying the coordinate head. The norm is logged so this can be watched.

AdamW with amsgrad, lr = 10^{-4} , weight decay 10^{-12} .

The decay is effectively off; it is present because EDM uses this exact configuration, and diffusion training is known to be sensitive to regularisation that fights the noise-prediction objective.

EMA of the weights.

Diffusion sampling chains T forward passes, so small weight noise compounds. EMA keeps a shadow copy updated as $\theta_{\text{ema}} \leftarrow 0.999 \theta_{\text{ema}} + 0.001 \theta$ after every step, and it is the shadow that is sampled from. Its horizon is ~ 1000 steps, under one epoch at 1562 iterations/epoch.

The shadow is a whole diffusion model.

EMA deep-copies the EquivariantDiffusion object, so `ema.shadow.sample(...)` works directly. The averaging loop iterates over `state_dict()` values and skips non-floating-point entries, so integer buffers are copied rather than blended.

6.2 Per-epoch evaluation

Each epoch records train loss, validation loss and wall time; every `--eval-every` epochs it additionally *generates* molecules and scores them. The size-to-mask conversion is the interesting shape step:

```
1 sizes = torch.multinomial(size_hist, n_samples, replacement=True) # [B]
2 mask = (torch.arange(max_n)[None, :] < sizes[:, None]).float() # [B,N]
3 x, one_hot, _ = model.sample(mask) # [B,N,3], [B,N,K]
4 mols = to_molecules(x, one_hot, mask, symbols) # dense -> list of (n_i,3) + symbols
5 return evaluate(mols, implicit_h), mols
```

Listing 10: `sample_and_score`

`to_molecules` performs the inverse of the dataset’s padding: it argmaxes the one-hot back to indices `[B,N]`, converts the mask to a boolean, and uses it to slice each molecule down to its true length, yielding a ragged Python list of $(n_i, 3)$ arrays with matching element symbols. Checkpoints store both raw and EMA weights, the config, and `size_hist`, so `sample.py` can reconstruct the model without being told the architecture.

7 Sampling: `EquivariantDiffusion.sample`

7.1 The reverse process

Sampling runs the chain backwards from pure noise. The generative model is factorised as $p(\mathbf{z}_0) = p(\mathbf{z}_T) \prod_{t=1}^T p(\mathbf{z}_{t-1} | \mathbf{z}_t)$, and each reverse step is obtained by taking the true posterior $q(\mathbf{z}_s | \mathbf{z}_t, \mathbf{z}_0)$ for $s = t - 1$ and substituting the network’s estimate of $\mathbf{z}_0$. Define

$$\alpha_{t|s} = \frac{\alpha_t}{\alpha_s}, \quad \sigma_{t|s}^2 = \sigma_t^2 - \alpha_{t|s}^2 \sigma_s^2. \quad (9)$$

The Gaussian posterior is

$$q(\mathbf{z}_s | \mathbf{z}_t, \mathbf{z}_0) = \mathcal{N}\left(\frac{\alpha_{t|s}\sigma_s^2}{\sigma_t^2} \mathbf{z}_t + \frac{\alpha_s\sigma_{t|s}^2}{\sigma_t^2} \mathbf{z}_0, \frac{\sigma_{t|s}^2\sigma_s^2}{\sigma_t^2} \mathbf{I}\right). \quad (10)$$

Substituting $\mathbf{z}_0 = (\mathbf{z}_t - \sigma_t \epsilon) / \alpha_t$ and collecting the $\mathbf{z}_t$ terms,

$$\alpha_{t|s}\sigma_s^2 + \frac{\sigma_{t|s}^2}{\alpha_{t|s}} = \frac{\alpha_{t|s}^2\sigma_s^2 + \sigma_t^2 - \alpha_{t|s}^2\sigma_s^2}{\alpha_{t|s}} = \frac{\sigma_t^2}{\alpha_{t|s}}, \quad (11)$$

so the mean collapses to a two-term expression in $\mathbf{z}_t$ and the predicted noise, which is exactly what the code computes:

$$\boldsymbol{\mu}_{t \rightarrow s} = \frac{\mathbf{z}_t}{\alpha_{t|s}} - \frac{\sigma_{t|s}^2}{\alpha_{t|s}\sigma_t} \hat{\epsilon}, \quad \text{std} = \frac{\sigma_{t|s}\sigma_s}{\sigma_t} \quad (12)$$

```
1 s_int = t_int - 1
2 a_t, s_t = self.alphas[t_int], self.sigmas[t_int] # 0-dim tensors (scalars)
3 a_s, s_s = self.alphas[s_int], self.sigmas[s_int]
4
5 a_ts = a_t / a_s
6 var_ts = (s_t**2 - a_ts**2 * s_s**2).clamp(min=1e-12)
7 std = (var_ts.sqrt() * s_s / s_t).view(1, 1, 1) # -> [1,1,1], broadcasts
8
9 t = torch.full((b,), t_int / self.timesteps) # [B]
10 pred_x, pred_h = self.dynamics(t, z_x, z_h, node_mask) # [B,N,3], [B,N,K]
11
12 coef = (var_ts / (a_ts * s_t)).view(1, 1, 1) # [1,1,1]
```

```

13 mu_x = z_x / a_ts - coef * pred_x # [B,N,3]
14 mu_h = z_h / a_ts - coef * pred_h # [B,N,K]
15
16 z_x = mu_x + std * sample_zero_com_noise(z_x.shape, node_mask, z_x.device)
17 z_h = mu_h + std * torch.randn_like(z_h) * node_mask.unsqueeze(-1)
18 return remove_mean(z_x, node_mask), z_h * node_mask.unsqueeze(-1)

```

Listing 11: `_p_step` — one ancestral step $\mathbf{z}_t \rightarrow \mathbf{z}_{t-1}$

Unlike training, sampling uses *one* timestep for the whole batch, so α, σ are 0-dimensional tensors reshaped to $[1,1,1]$ rather than $[B,1,1]$. The coordinate noise is again drawn in $\mathcal{X}_0$, and `remove_mean` is reapplied at the end of every step so floating-point drift cannot accumulate over 1000 iterations.

7.2 The chain and the final decode

```

1 z_x = sample_zero_com_noise((b, n, 3), node_mask, device) # [B,N,3]
2 z_h = torch.randn(b, n, self.num_types, device=device) * nm # [B,N,K]
3
4 for t_int in range(self.timesteps, 0, -1): # T .. 1
5     z_x, z_h = self._p_step(z_x, z_h, t_int, node_mask)
6
7 t = torch.zeros(b, device=device) # [B]
8 pred_x, pred_h = self.dynamics(t, z_x, z_h, node_mask)
9 a0, s0 = self.alphas[0], self.sigmas[0]
10 x = remove_mean((z_x - s0 * pred_x) / a0, node_mask) # [B,N,3]
11 h = (z_h - s0 * pred_h) / a0 # [B,N,K]
12
13 x, h = self.unnormalize(x, h) # h /= 0.25
14 types = h.argmax(-1) # [B,N,K] -> [B,N]
15 one_hot = F.one_hot(types, self.num_types).float() * nm # [B,N] -> [B,N,K]
16 return x * nm, one_hot, frames

```

Listing 12: `sample` — the full reverse trajectory

The loop runs T times and each iteration is one full network forward pass — sampling costs $1000\times$ a training forward pass, which is why generating a few hundred molecules takes longer than an epoch of training on them.

Two steps close the process:

1. **A final denoise at $t = 0$.** Because $\sigma_0 = 0.0032 \neq 0$, $\mathbf{z}_0$ is still very slightly noisy; $\mathbf{x} = (\mathbf{z}_x - \sigma_0 \hat{\epsilon}_x) / \alpha_0$ removes that last increment.
2. **Discretisation of the atom types.** `argmax` over the last axis collapses $[B,N,K]$ to $[B,N]$ integer labels, which are re-expanded to a clean one-hot. This is the point at which the continuous relaxation becomes a discrete chemical decision — and where information is irrecoverably thrown away, since a near-tie between carbon and nitrogen is resolved silently.

7.3 A closed-form check of the sampler

The reverse-process algebra above can be validated without any trained network. If the data were $\mathbf{x} \sim \mathcal{N}(0, \mathbf{I})$, then since $\mathbf{z}_t \sim \mathcal{N}(0, (\alpha_t^2 + \sigma_t^2)\mathbf{I}) = \mathcal{N}(0, \mathbf{I})$, the optimal denoiser is available in closed form:

$$\mathbb{E}[\epsilon \mid \mathbf{z}_t] = \sigma_t \mathbf{z}_t. \quad (13)$$

Substituting this exact denoiser for the network must return samples with unit variance — any error in $\alpha_{t|s}$, $\sigma_{t|s}^2$ or the standard deviation shows up as a drift in scale. The test in `test_smoke.py`

does precisely this with $n = 6$ atoms and recovers a standard deviation of **0.911** against the analytic prediction for the projected subspace,

$$\sqrt{\frac{n-1}{n}} = \sqrt{5/6} = 0.913. \quad (14)$$

The agreement, including the subspace correction, is the evidence that the schedule and the ancestral step are implemented correctly.

8 Evaluation: [qm9_diffusion/stability.py](#)

The model outputs coordinates, not bonds, so the metric has to infer chemistry from geometry. For each unordered pair, the interatomic distance is compared against tabulated single/double/triple bond lengths for that element pair, with tolerances of (10, 5, 3) pm; the resulting bond orders are summed per atom to give an inferred valence.

Metric	Definition	Shape reduction
atom stability	inferred valence = expected valence	$(n, 3) \rightarrow n$ booleans $\rightarrow$ scalar
molecule stability	<i>all</i> atoms in the molecule stable	n booleans $\rightarrow$ 1 boolean

Expected valences are H:1, C:4, N:3, O:2, F:1. Molecule stability is the harsher metric by construction — one wrong atom in 18 fails the whole molecule — so it lags atom stability substantially in an undertrained model.

Validating the metric itself. Run on *real* QM9 geometries the implementation gives 0.994 atom / 0.951 molecule stability, against the published 0.99/0.95. Reproducing the reference numbers on reference data is what establishes that the tolerances and bond tables are right, so that a poor score on generated molecules can be attributed to the model.

The heavy-atom caveat. With `--remove-h`, exact valence is unsatisfiable: a carbon that came from methane has no heavy-atom neighbours at all. In that mode the check is relaxed to *valence* $\leq$ *allowed*, treating the deficit as implicit hydrogens. On real data this reads 0.996/0.978. It is a much weaker test, and no-H numbers must not be compared with with-H ones.

9 Masking: the invariant to preserve

Because the implementation is dense, correctness rests on padded slots never influencing anything. The places that enforce it:

Location	Mechanism	What breaks without it
dataset <code>__getitem__</code>	<code>* mask[:, None]</code>	padded rows decode as spurious hydrogens
<code>remove_mean</code>	masked mean, divide by n_b	centre of mass computed over N , not n_b
<code>edge_mask</code>	outer product of masks, minus I	phantom atoms send messages
<code>n_edges</code>	<code>edge_mask.sum(2)</code>	aggregation normalised by $N - 1$
every layer output	<code>* node_mask</code>	padding accumulates state across layers
loss	<code>eps</code> masked at creation	padded slots contribute error
dof	<code>node_mask.sum()</code>	per-dimension mean depends on padding

`test_masking_and_com` asserts the output invariant directly: for a batch containing molecules of sizes 9, 7, 5, 3 padded to $N = 9$, both output tensors are 0 in every padded slot, and the masked centre of mass is zero to 1.5×10^{-11} .

10 Computational cost

10.1 Where the memory goes

The dense formulation materialises $B \times N^2$ edges, each carrying H channels. For $B = 64$, $N = 29$, $H = 192$ in float32:

Tensor	Elements	Memory
edge_mlp input $[B, N, N, 2H+1]$	20.7 M	82.9 MB
message m $[B, N, N, H]$	10.3 M	41.3 MB
node tensor $[B, N, H]$	0.36 M	1.4 MB

Several such activations are retained per layer for the backward pass, so training memory scales as $\mathcal{O}(L \cdot B \cdot N^2 \cdot H)$ — the edge tensor, not the parameters (1.6 M, about 6 MB), is what fills the device. Reduce `--batch-size` or `--hidden` first when memory is tight.

For QM9 the dense choice costs almost nothing: a fully connected 29-atom molecule has 812 directed edges, and a sparse implementation of the same model would build essentially the same number. Dense wins on simplicity here; it would not for larger molecules.

10.2 Measured throughput

On Apple Silicon (MPS), batch size 64, one epoch = 100k molecules:

Configuration	Params	ms/iter	min/epoch
$H=256$, $L=9$ (EDM paper)	4.22 M	613	16.0
$H=192$, $L=6$ (default)	1.60 M	294	7.7
$H=128$, $L=4$	0.48 M	123	3.2
$H=128$, $L=4$, <code>--remove-h</code>	0.48 M	21	0.5

The last two rows are the same model on the same number of molecules; only N changed, from 29 to 9. The measured speedup is $123/21 \approx 5.9\times$, against the $(29/9)^2 \approx 10.4\times$ that pure N^2 scaling of the edge axis would predict — the shortfall is per-iteration overhead (kernel launches, the optimiser step, host-device synchronisation) that does not shrink with N . It is nevertheless the reason the heavy-atom mode is the practical setting for a laptop.

11 Summary: one forward pass end to end

Collecting every shape change for a single training step with the default configuration $B=64$, $N=29$, $K=5$, $H=192$, $L=6$:

#	Operation	Shape	Where
1	batch from DataLoader	$[64, 29, 3], [64, 29, 5], [64, 29]$	data.py
2	scale types by 0.25	unchanged	normalize
3	project coordinates into $\mathcal{X}_0$	unchanged	remove_mean
4	draw one timestep per molecule	$[64]$	loss
5	gather α_t, σ_t	$[64] \rightarrow [64, 1, 1]$	loss
6	add noise	$[64, 29, 3], [64, 29, 5]$	loss
7	embed types + time	$[64, 29, 6] \rightarrow [64, 29, 192]$	embed
8	build the edge mask	$[64, 29, 1] \rightarrow [64, 29, 29, 1]$	forward
9	$\times L=6$: lift to edges	$[64, 29, 29, 385]$	edge_mlp
10	message	$[64, 29, 29, 192]$	edge_mlp
11	coordinate scalar	$[64, 29, 29, 1]$	coord_mlp
12	contract to nodes	$[64, 29, 192], [64, 29, 3]$	sum over axis 2
13	coordinate displacement, re-projected	$[64, 29, 3]$	forward
14	decode type noise	$[64, 29, 192] \rightarrow [64, 29, 5]$	decode
15	squared error / d.o.f.	$\rightarrow$ scalar	loss

At sampling time steps 7–14 are repeated $T = 1000$ times with the shapes unchanged, followed by an `argmax` that collapses $[B, N, K]$ to $[B, N]$ and a final one-hot re-expansion.

References

1. E. Hoogeboom, V. G. Satorras, C. Vignac, M. Welling. *Equivariant Diffusion for Molecule Generation in 3D*. ICML 2022. <https://arxiv.org/abs/2203.17003>
2. V. G. Satorras, E. Hoogeboom, M. Welling. *E(n) Equivariant Graph Neural Networks*. ICML 2021. <https://arxiv.org/abs/2102.09844>
3. J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020. <https://arxiv.org/abs/2006.11239>
4. R. Ramakrishnan, P. O. Dral, M. Rupp, O. A. von Lilienfeld. *Quantum chemistry structures and properties of 134 kilo molecules*. Scientific Data 1, 140022 (2014).